



SafeWay

Rapport de Projet Génie Logiciel :

Matière : Génie Logiciel

Nom de la tutrice : Madame Sirine Hawari

Sujet : Développer une application de simulation de déplacement d'agents dans un bâtiment lors d'une évacuation dans un cas d'urgence

Thématique n°3 : Déplacement d'agents dans un graphe

Filière : ING1 Génie Informatique groupe GI05

Réalisé par le Groupe GI05-B :

- MEHANNI Rayane
- MURUGAN Alexis
- OZSOLAK Atahan
- POLAT-DEMIRKAYA Erwan
- SAOULI Rémi

Établissement : CY Tech, avenue du Parc, 95000, Cergy

Année universitaire : 2025/2026

Sommaire

Introduction.....	3
Technologies utilisées.....	4
Planning.....	6
Semaine du 18/05.....	6
Semaine du 25/05.....	7
Semaine du 01/06.....	7
Semaine du 08/06.....	8
Diagrammes UML.....	9
Diagramme de classes.....	9
Diagramme de cas d'utilisation.....	9
Difficultés.....	10
Conclusion.....	11

Introduction

Lors d'évacuations en cas d'urgence, surtout dans les grands bâtiments dans lesquels on peut trouver plusieurs occupants, plusieurs victimes succombent à cause des bousculades qui peuvent être mortelles lors des déplacements, ainsi qu'à la saturation des portes, qui donne lieu à des comportements humains empirant la situation. C'est dans cette perspective que notre solution a vu le jour : SafeWay, un dispositif permettant de guider les occupants d'un bâtiment vers les sorties les plus optimisées tout en évitant la saturation des portes.

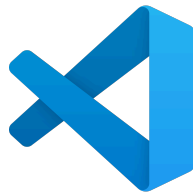
Notre projet en génie logiciel vient en soutien à notre solution afin de pouvoir simuler notre dispositif et ainsi observer le facteur dont dépend le plus ce dernier : le facteur humain. Le projet nous permet de faire une analyse sur différents échantillons avec plusieurs visuels, de sorte à observer les déplacements en temps réel des occupants et leurs interactions entre eux et avec les destinations.

Ce projet a été réalisé par notre groupe GI5-B et constitue la suite de notre projet en éthique et en design.

Technologies utilisées

Pour développer notre projet en génie logiciel, qui consiste en une simulation de déplacement d'agents dans un graphe, nous nous sommes appuyés sur les technologies suivantes :

- **VS Code** : Un éditeur de code source gratuit et open-source. C'est un outil très puissant et polyvalent qui permet de coder dans plusieurs langages différents.



- **Git** : Un logiciel de gestion de versions décentralisé. Il est gratuit et permet de gérer les ajouts et changements apportés au code source de manière tracée.



- **GitHub** : Une plateforme d'hébergement publique et gratuite qui permet de stocker, se partager et travailler sur des projets en groupe.



- **Java** : Un langage de programmation de haut niveau orienté objet permettant entre autres de développer des logiciels.



- **JavaFx** : Un framework qui permet de créer des applications Java dotées d'une interface utilisateur moderne hautement portable.



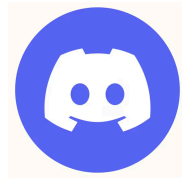
- **Maven** : Un outil de gestion et d'automatisation de production des projets logiciels Java, utilisé pour automatiser l'intégration continue lors d'un développement de logiciel.



- **Jira** : Un système de gestion de projet permettant à des équipes de travailler ensemble.



- **Discord** : Une plateforme de communication qui permet aux utilisateurs d'interagir par texte, voix et vidéo.



Planning

Sans compter le travail sur notre projet côté éthique et design, nous avons travaillé pendant quatre semaines sur le volet technique en équipe, accompagnés par notre tutrice, Madame **Sirine Hawari**. Notre travail sur cette période s'est déroulé de la manière suivante :

Semaine du 18/05

Dès que nous avons eu accès au cahier des charges et à la liste de nos tuteurs, nous nous sommes élançés pour les lire et commencer à visualiser le projet. Après cela, nous avons contacté notre tutrice attitrée pour organiser une première réunion afin de discuter de la marche à suivre ainsi que des questions auxquelles nous avons réfléchi lors de la lecture du cahier des charges. La réunion s'est déroulée impeccablement, notamment parce que nous étions préparés et que la tutrice nous a directement donné les prochaines étapes à suivre afin de pouvoir compléter le projet et d'avoir un rendu adéquat. À la fin de la réunion, la tutrice nous a aidés à nous organiser avant de nous lancer dans le projet en mettant en valeur plusieurs outils importants afin de pouvoir compléter un MVP démontrable pour la prochaine réunion.

Pour ce qui est du partage des tâches et du suivi de ces dernières, nous avons mis en commun un projet Jira, qui est un logiciel permettant de garder une trace des différentes tâches au cours d'un projet, de leur état et des personnes auxquelles elles sont assignées, afin de savoir en temps réel qui travaille sur quoi et ce qui doit être fait. Cette façon d'organiser les tâches nous a permis d'aller plus rapidement en évitant les complications liées à l'identification des fonctionnalités à développer, ce qui nous a permis d'avancer plus efficacement. Le fait d'avoir créé un dépôt GitHub nous a aussi facilité la tâche, nous permettant de partager notre code tout en gardant un historique des versions. De cette façon, chacun peut travailler sur les parties qui le concernent sans pour autant retarder les autres.

Pour ce qui est de l'organisation au sein de l'équipe, nous nous sommes divisés les tâches en fonction des affinités et des envies de chacun. Rayane et Erwan ont décidé de s'occuper du backend, en créant les modèles qui contiennent les classes métier et la logique métier dont a besoin l'application en termes de nœuds, d'agents, d'arêtes et de simulation. Alexis, Atahan et Rémi, quant à eux, se sont occupés du côté frontend ainsi que du côté contrôleur pour la liaison entre la logique métier et l'interface graphique. Grâce à cela, nous avons pu nous répartir les tâches de façon homogène sur différents aspects du projet afin d'avancer en même temps et efficacement sous une architecture MVC (Model - View - Controller). Après cela, toute l'équipe a décidé de commencer à

travailler sur ses tâches après les avoir ajoutées sur Jira afin d'entamer l'implémentation du projet.

Semaine du 25/05

La semaine d'après s'est surtout consacrée au début de l'implémentation du MVP que nous devons montrer à la tutrice lors de la prochaine réunion. Rayane a commencé en créant l'architecture des fichiers pour que chacun puisse travailler sur des fichiers différents et a commencé à créer les classes de logique métier. Il a développé les éléments essentiels d'un graphe, notamment les nœuds et les arêtes, ainsi que les agents qui s'y trouvent. Puis, il a ajouté une classe de simulation ainsi qu'un déplacement simple des agents suivant le plus court chemin. Erwan l'a aidé du côté du backend sur certaines classes et les relations entre elles.

Pendant ce temps, Rémi avait commencé à développer ses connaissances en JavaFx afin de pouvoir afficher les nœuds sur une page. Pour ce qui est de leur affichage, nous avons décidé de les disposer de façon circulaire afin de permettre aux arêtes de rester visibles sans se chevaucher malgré un grand nombre de nœuds. Alexis, lui, travaillait sur l'affichage des arêtes entre deux nœuds avec des flèches dirigées afin d'ajouter ces fonctionnalités à la page de Rémi. Il s'est aussi occupé de l'aspect visuel pour que les pages aient une direction artistique plus proche des maquettes que nous avons réalisées en design. Atahan, quant à lui, avait créé les pages de détails des différents éléments, notamment les nœuds, les arêtes et les agents, afin de permettre, après un clic sur l'un d'eux, de visualiser ses informations et ses caractéristiques.

Après que chacun eut terminé sa partie, nous avons intégré les fonctionnalités de chacun afin d'obtenir une première version de test qui nous a permis de visualiser ce qui fonctionnait et ce qui présentait des bugs. Grâce à cela, toutes les fonctionnalités implémentées par la suite ont été rapides à tester. Nous avons clôturé cette première partie du projet avec un diagramme de classes mettant en valeur toutes les classes utilisées dans notre projet ainsi que les relations entre elles.

Puis le jour de la deuxième réunion avec notre tutrice est arrivé, et c'est à ce moment-là que nous lui avons présenté notre MVP. Elle nous a donné de nombreuses remarques sur les éléments qui devaient être améliorés ainsi que sur les fonctionnalités qui devaient être ajoutées. Nous avons donc pris note de tout cela et nous étions satisfaits d'être relativement avancés, avec une version MVP du projet disposant déjà d'une interface graphique fonctionnelle.

Semaine du 01/06

Ensuite, chacun s'est penché sur les fonctionnalités à ajouter qui le concernaient, ainsi que sur les bugs qui devaient être corrigés. Côté logique métier, plusieurs changements ont été effectués, notamment dans la gestion des lieux (nœuds ou arêtes) et leur façon de stocker les agents. Les déplacements de ces derniers ont également été modifiés de sorte à ce que chaque état d'un agent corresponde à une manière de déterminer sa prochaine destination. Une personne paniquée, par exemple, suivra la foule, alors qu'une personne calme privilégiera plutôt le chemin le plus court afin d'atteindre la sortie plus rapidement. Pour ce qui est du déplacement au sein d'une arête, la vitesse de l'agent est exponentiellement décroissante en fonction de la congestion de cette dernière. De cette façon, une arête saturée causera davantage de bousculades, rendant le déplacement des agents plus difficile. Un système de fichiers a aussi été implémenté, permettant de sauvegarder l'état actuel de la simulation dans un fichier binaire dont le nom correspond à celui donné à la simulation. Ces sauvegardes peuvent ensuite être restaurées comme autre manière de lancer une simulation, au lieu d'en créer une nouvelle.

Du côté de l'interface graphique, beaucoup d'indicateurs visuels ont été ajoutés afin que la simulation soit claire, notamment au niveau de la coloration des nœuds, des arêtes et de leur positionnement. Plusieurs autres pages ont été créées pour permettre la création de nouveaux éléments (nœuds, arêtes ou agents) avec des attributs spécifiques. Comme auparavant, nous avons fait en sorte que la simulation puisse être exécutée par étapes et que l'on puisse passer de l'une à l'autre par un simple clic. Nous avons conservé cette fonctionnalité et en avons ajouté une autre où les mouvements sont temporels, permettant ainsi de visualiser la simulation en fonction du temps. Grâce à cela, nous avons implémenté l'essentiel des fonctionnalités nécessaires pour lancer et visualiser une simulation de déplacement d'agents dans un graphe.

Après avoir montré cette version avancée à notre tutrice, elle nous a donné des indications concernant le rapport ainsi que les diagrammes que nous devons préparer, en plus de quelques modifications UI/UX à entreprendre.

Semaine du 08/06

La dernière semaine a surtout été consacrée aux dernières modifications nécessaires à notre projet. Du côté de la logique métier, de la documentation a été ajoutée afin de pouvoir générer une Javadoc, et une factorisation du code a été effectuée pour simplifier ce dernier. L'objectif était de rendre le code clair afin qu'il puisse être facilement compréhensible par n'importe quelle personne. Du côté de l'interface graphique, quelques modifications ont été apportées, principalement au niveau du style, afin de permettre une meilleure visualisation du graphe et de ses éléments.

Après cela, nous nous sommes mis à travailler sur les diagrammes de classes et de cas d'utilisation de notre projet, ainsi que sur le README et le rapport afin de le finaliser. Une fois ces éléments terminés, nous avons organisé une dernière réunion avec notre tutrice afin de vérifier que tout était conforme et que rien n'avait été oublié.

Diagrammes UML

À la fin de notre projet, nous avons réalisé plusieurs diagrammes UML afin de comprendre, d'une part, la structure du code et de l'architecture technique et, d'autre part, les différentes utilisations possibles de notre application, ainsi que les fonctionnalités qu'elle permet d'offrir et les résultats qu'elle peut produire.

Diagramme de classes

Diagramme de cas d'utilisation

Difficultés

Lors de la réalisation de ce projet en génie logiciel, la plus grande difficulté que nous avons rencontrée était la gestion du temps. Entre les rattrapages, les emplois du temps de chacun et les procédures de mobilité internationale pour l'ING2, il était très difficile de regrouper à chaque fois tout le groupe pour une réunion d'équipe. Nous avons quand même pu pallier cela en multipliant les réunions et en communiquant régulièrement.

Le temps a également été un défi, donc nous n'avons pas eu l'occasion d'implémenter plusieurs fonctionnalités bonus auxquelles nous avons réfléchi. Au vu du temps qu'il nous restait, nous avons donc décidé d'implémenter uniquement les fonctionnalités demandées, tout en faisant attention à ce que l'application fonctionne parfaitement, sans bugs ni problèmes lors de l'exécution.

Le dernier problème que nous avons rencontré est survenu au début du projet concernant la partie interface graphique avec JavaFx. Comme nous n'avions pas forcément d'idée de la manière dont sont gérés les projets JavaFx, nous avons été bloqués un certain temps à réfléchir à la façon d'initier le projet, de sorte à éviter d'être bloqués à mi-chemin. Cependant, grâce à nos recherches et à nos concertations, nous avons réussi à mettre en place une bonne architecture qui nous a permis de faciliter le développement de l'application tout au long du processus.

Conclusion

Ce projet nous a permis de découvrir une autre facette de ce type de projet qu'est le génie logiciel, en apprenant les concepts et en comprenant comment les applications fonctionnent et sont développées avec une architecture MVC (Model - View - Controller) et une pensée orientée objet (Java).

Notre tutrice, quant à elle, nous a permis de nous mettre en situation de monde professionnel en respectant plusieurs deadlines tout au long du projet, ainsi qu'en utilisant des outils très utilisés au sein des entreprises comme Jira, qui permet de faciliter l'assignation des tâches et le travail au sein des équipes, sans oublier les différents termes techniques de ce secteur que nous avons pu apprendre.

Enfin, cette application nous a permis de développer nos compétences en programmation orientée objet en premier plan, en s'appuyant sur la pensée objet lors de l'implémentation de la logique métier, en plus d'utiliser un framework (JavaFX) pour créer des interfaces graphiques modernes, permettant de visualiser de façon intuitive notre système.